

Real-Time Pothole Detection Based on Vision-Dominant Sensor Fusion and Dual ESP32 Architecture

Sanskar Tiwari^{1*}, Swarali Bansod², Eshwari Kognole³ and Shruti Shinde⁴

1,2,3,4 Department of Electronics and Telecommunication Engineering, Pune Institute of Computer Technology (PICT), Pune, Maharashtra, India

* Correspondence: sanskartiwari.smt@gmail.com

S.T.: sanskartiwari.smt@gmail.com; S.B.: swaralibansod@gmail.com; E.K.: eshwarik116@gmail.com; S.S.: shrutishinde049@gmail.com

Abstract: Potholes are a persistent and costly threat to road safety, vehicle integrity, and urban infrastructure. Traditional road inspection methods: manual surveys or expensive profilometer vehicles: are slow, unreliable, and difficult to scale. This paper presents the Intelligent Pothole Detection System (IPDS), a low-cost, vehicle-mounted IoT platform that automatically identifies and rates road hazards in real time. The system uses a dual ESP32 architecture that keeps video capture and inertial sensing on separate microcontrollers, so neither task slows the other down. A custom-trained YOLOv8m model runs computer vision inference on a central Python hub, while the SORT tracking algorithm and geometric heuristic filters reduce false positives. The model achieves a mean average precision (mAP@0.5) of 81.68%, mAP@0.5:0.95 of 55.95%, precision of 82.33%, and recall of 74.42% on the validation split. When a pothole is visually confirmed, an HTTP request directs the sensor node to capture MPU6050 accelerometer data, NEO-6M GPS coordinates, and a DS3231 RTC timestamp. A weighted fusion formula then combines visual confidence with peak jerk magnitude to produce a normalized pothole severity index. Field tests show 10–15 FPS inference on a CPU-only host and high-precision geo-tagged CSV logs ready for GIS mapping and municipal maintenance planning. The complete hardware costs under Rs. 2,500 per unit.

Keywords: *pothole detection; sensor fusion; YOLOv8; ESP32; edge computing; SORT tracking; Internet of Things; road infrastructure monitoring*

1. INTRODUCTION

Road infrastructure underpins both economic productivity and public safety in urban environments. Potholes the most common and immediately dangerous form of road surface degradation develop through a cyclical failure process: subsurface water weakens load-bearing soil, repeated vehicle loads fatigue the asphalt, and thermal expansion and contraction open cracks until the pavement collapses inward [13]. Once a pothole forms, the consequences accumulate rapidly. Suspension and tire damage costs billions of dollars each year; drivers swerving to avoid potholes cause phantom traffic jams; and two-wheeled road users risk sudden loss of control, making unaddressed potholes a lethal threat to vulnerable road users such as motorcyclists and cyclists [2].

Nevertheless, the dominant approach to road quality assessment remains reactive. Municipalities wait for citizen complaints or deploy expensive specialized vehicles laser profilometers or ground-penetrating radar units that survey a given road segment only once every two to five years [13]. That long survey gap allows minor surface defects to grow into serious hazards undetected, and repair crews are often dispatched without any objective prioritization of which hazards pose the greatest risk. The logistical inefficiency means a crew might travel to a minor reported depression while a far more dangerous pothole nearby goes unaddressed.

Recent advances in edge computing, low-power microcontrollers, and open-source computer vision have opened an alternative path: passive, continuous, and automated road monitoring built into everyday fleet vehicles. A garbage truck or postal delivery van fitted with an inexpensive detection unit could survey an entire city’s road network repeatedly and passively at a fraction of the cost of traditional methods [2]. The key design challenge is producing a system cheap enough for fleet-wide deployment, robust enough to reject the wide variety of visual and inertial false positives encountered on real roads, and accurate enough to generate actionable maintenance data.

To address these challenges, this paper introduces the Intelligent Pothole Detection System (IPDS): a dual-microcontroller IoT platform that fuses deep learning-based computer vision with inertial sensor telemetry for real-time, geo-tagged, severity-scored pothole detection. The main contributions of this work are as follows:

1. A dual ESP32 hardware architecture that keeps video acquisition and inertial sensing on separate microcontrollers, eliminating frame-rate degradation caused by I2C blocking latency on the camera node.
2. A vision-dominant sensor fusion method in which YOLOv8 computer vision generates detection hypotheses and the MPU6050 accelerometer provides physical validation, forming a logical AND gate that eliminates single-modality false positives.
3. A multi-stage false positive suppression pipeline combining confidence thresholding, Kalman-filtered SORT tracking, and geometric heuristic filters for bounding box area, aspect ratio, and object persistence.
4. A validated YOLOv8m model achieving 81.68% mAP@0.5, 82.33% precision, and 74.42% recall on a publicly available road-surface dataset.
5. A normalized severity index that mathematically combines the YOLOv8 confidence score with peak inertial jerk magnitude for objective hazard prioritization.
6. A complete, deployable system architecture under Rs. 2,500 per unit that generates Firebase-ready CSV logs with GPS coordinates, timestamps, and severity scores.

The rest of this paper is organized as follows. Section 2 reviews related work. Section 3 describes the system architecture and hardware design. Section 4 details the firmware implementation. Section 5 covers the computer vision and tracking pipeline. Section 6 presents the sensor fusion and severity model. Section 7 discusses results and limitations. Section 8 concludes with directions for future work.

2. RELATED WORK

2.1. Vision-Based Pothole Detection

Early computer vision approaches to pothole detection relied on hand-crafted features: edge detectors, color histogram thresholding, and disparity maps from stereo cameras [1]. While effective in controlled settings, these methods degrade quickly under real-world conditions where potholes may be filled with rainwater, coated in dust, or hidden by shadows cast by surrounding structures.

Subsequently, deep convolutional neural networks (CNNs) greatly improved detection robustness. Fan et al. [3] showed that disparity-transformation-based detection could reliably distinguish normal pavement from surface depressions. Single-stage YOLO-family detectors have since achieved real-time inference speeds suitable for mobile deployment. Saluky et al. [4] evaluated YOLOv8 for urban pothole detection

and found mAP scores competitive with heavier two-stage detectors at a fraction of the computational cost. Similarly, Alzamzami et al. demonstrated a YOLOv8 deployment achieving an F1-score of 95%, underscoring the architecture’s suitability for mobile road inspection platforms [5]. More recently, Yurdakul and Guller [6] extended YOLOv8-Nano with Dynamic Snake Convolution and attention modules, reaching 93.7% precision and 90.4% recall on a custom RGB-D dataset while remaining lightweight enough for real-time embedded deployment.

2.2. Accelerometer-Based Detection

A parallel line of research mounts inertial measurement units (IMUs) on vehicle chassis to detect the vertical acceleration spike caused by a tyre dropping into a road depression [8]. Mednis et al. [7] were early pioneers of smartphone-based accelerometer logging for pothole detection. Pure accelerometer approaches, however, suffer from high false positive rates: speed bumps, railroad crossings, and hard braking all produce nearly identical acceleration signatures. Additionally, spike magnitude depends heavily on the host vehicle’s suspension stiffness, making cross-vehicle comparison challenging. Consequently, more recent IoT-based work has demonstrated sensor networks using dual-accelerometer nodes to improve reliability by requiring confirmation from both sensors before logging an event [9].

2.3. Sensor Fusion Approaches

Combining vision and inertial data has been explored by several groups. Koch and Brilakis [10] fused GPS traces with image-based crack detection for infrastructure mapping. Similarly, Tedeschi and Benedetto [11] proposed pairing camera data with vehicle dynamics for pavement assessment. Multi-sensor IoT architectures combining LiDAR, accelerometers, and ultrasonic sensors with GPS tagging have also been proposed for classifying road surfaces in real time [12]. Nevertheless, all prior implementations operate at significantly higher hardware cost. The IPDS, by contrast, builds on this body of work by implementing fusion at the embedded microcontroller level through an event-triggered HTTP protocol, enabling real-time decisions without cloud dependency at a hardware cost one to two orders of magnitude below prior implementations.

2.4. Tracking and Duplicate Prevention

Object tracking across frames is essential to prevent the same pothole from being counted multiple times. The Simple Online and Realtime Tracking (SORT) algorithm [14] uses the Hungarian algorithm for data association and a Kalman filter for state prediction, providing a lightweight solution well suited to CPU-only inference environments. The IPDS therefore integrates SORT as its core deduplication mechanism.

3. SYSTEM ARCHITECTURE

3.1. Architecture Overview

The IPDS operates around an event-driven, distributed computing model that separates hardware responsibilities across three physical tiers: two constrained ESP32 microcontrollers at the acquisition edge, a Python processing hub performing AI inference, and a file-system output layer generating CSV logs and annotated video.

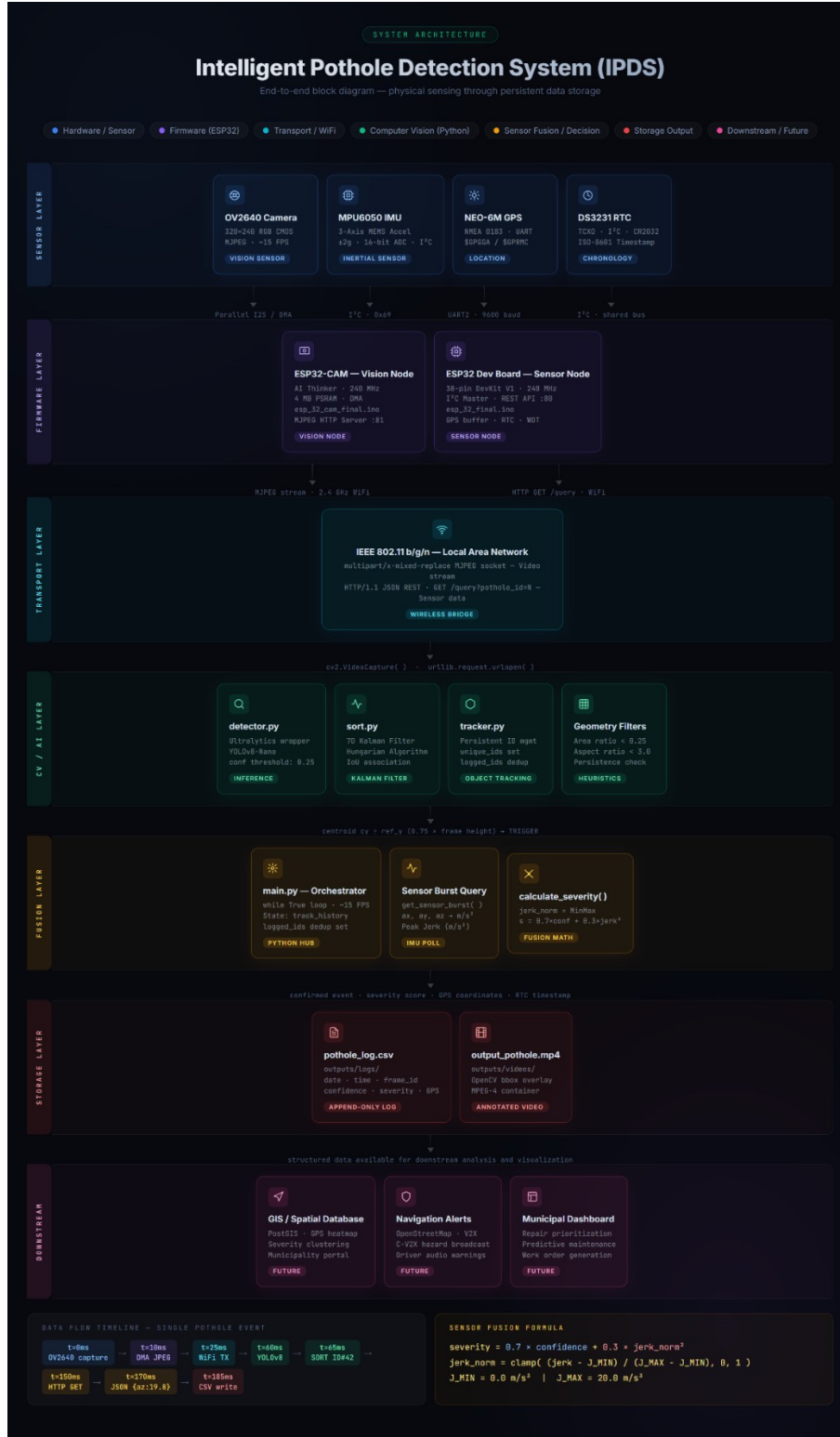


Fig. 1. IPDS system architecture. The hardware layer (blue) feeds data via WiFi MJPEG and serial JSON into the processing layer (green/purple), where YOLOv8m inference, SORT tracking, geometric false-positive filtering, and IMU jerk detection run concurrently. Confirmed events pass through the sensor fusion engine to the output layer (orange), which writes geo-tagged CSV logs, renders annotated video overlays, and exports an interactive pothole heatmap.

The four primary system layers are:

1. Hardware Layer: Physical sensors that interact with the real world.
2. Firmware Layer: C++ code that drives microcontroller hardware peripherals.
3. Processing Layer: Python AI and fusion logic that converts raw sensor data into structured insights.
4. Application Layer: Final output generation (CSV logs and annotated MP4 video).

3.2. Dual ESP32 Design Rationale

The most critical architectural decision in the IPDS is separating the vision node and sensor node onto two distinct ESP32 microcontrollers. On the ESP32-CAM, each frame acquisition calls `esp_camera_fb_get()`, which blocks the CPU for the full duration of JPEG capture and compression. Running I2C bus reads, which themselves require several milliseconds of clock stretching inside the same execution context causes severe frame drops and video stuttering. Routing all inertial sensing to a dedicated second microcontroller therefore lets the camera act as a pure streaming device with no peripheral interference. This architectural split is the foundational reason the system sustains 10–15 FPS while simultaneously polling a 6-axis IMU, GPS receiver, and RTC.

3.3. Hardware Components

Table 1 summarizes the hardware components, their roles, and key specifications. The complete bill of materials costs under Rs. 2,500 per unit.

Table 1. Hardware Component Summary.

Component	Role	Key Specification
ESP32-CAM	Vision Node	240 MHz, OV2640, WiFi
ESP32 Dev Board	Sensor Node	I2C/UART Master
MPU6050	Inertial Sensing	6-axis, 16-bit ADC
NEO-6M GPS	Geo-tagging	50-channel, UART
DS3231 RTC	Timestamping	TCXO, I2C, battery backup
Li-ion Power Bank	Power Supply	5V regulated, >600 mA peak
Processing Hub	AI Inference	Laptop / Jetson Nano

3.4. Vision Node

The ESP32-CAM module, equipped with the OV2640 2-megapixel image sensor, operates purely as a wireless webcam. Its firmware continuously captures 320×240 RGB frames via the parallel DVP camera interface, compresses them to JPEG using the hardware encoder, and pushes them to connected clients over an open HTTP MJPEG stream (multipart/x-mixed-replace). No detection or inference happens on this node; any computation beyond frame capture would cause thermal throttling and frame loss. Camera initialization targets road conditions: JPEG quality 10, frame size QVGA (320×240), and vertical flip enabled. The camera enclosure mounts at a −35° pitch angle, which eliminates the sky from the frame and aligns the 0.75 Y-coordinate pixel threshold to approximately 1.0 metre directly in front of the vehicle bumper.

3.5. Sensor Node

The ESP32 Development Board manages all inertial, positional, and time sensing. It connects to the MPU6050 6-axis IMU and DS3231 RTC on a shared I2C bus (GPIO 21/22, SDA/SCL), and to the NEO-6M GPS receiver over UART. The node implements a REST API and idles in the `server.handleClient()` loop, drawing minimal power. Only when it receives an HTTP GET request does it execute an I2C burst read from the MPU6050, retrieve the buffered GPS NMEA sentence, and return a JSON payload. The MPU6050 AD0 pin connects to ground, setting the I2C address to 0x68. At boot, the firmware verifies hardware presence by pinging expected I2C addresses (0x68 for MPU6050, 0x57 for DS3231); if any device is absent, the system halts and prints a diagnostic message to prevent silent data corruption. Table 2 lists the complete pin-level wiring for all sensor-node peripherals, and Table 3 itemizes the corresponding bill of materials at the connector level.

Table 2. MPU6050, DS3231 RTC, and NEO-6M GPS connections to the ESP32 sensor node.

Module / Pin	ESP32 Pin	Signal
MPU6050 – VCC	3V3	3.3V Power
MPU6050 – GND	GND	Ground
MPU6050 – SCL	GPIO 22	I2C Clock
MPU6050 – SDA	GPIO 21	I2C Data
MPU6050 – AD0	GND	Address = 0x68
MPU6050 – INT		Not connected
DS3231 – VCC	3V3	3.3V Power
DS3231 – GND	GND	Ground
DS3231 – SCL	GPIO 22	I2C Clock (shared)
DS3231 – SDA	GPIO 21	I2C Data (shared)
DS3231 – BAT		CR2032 backup cell
DS3231 – SQW		Not connected
NEO-6M – VCC	3V3 / VIN	3.3V Power
NEO-6M – GND	GND	Ground
NEO-6M – TX	GPIO 16 (RX2)	UART Data Out
NEO-6M – RX	GPIO 17 (TX2)	UART Data In
NEO-6M – PPS		Not connected

I2C bus configuration: 4.7 k Ω pull-up resistors on both SDA and SCL lines (3.3 V rail), 100 kHz standard-mode bus speed, MPU6050 address 0x68 (AD0 tied to GND), DS3231 fixed address 0x57. UART2 operates at 9600 baud for NMEA sentence reception from the NEO-6M.

Table 3. Sensor-node bill of materials at the connector level.

Component	Module / Part	Qty	Connection / Notes
-----------	---------------	-----	--------------------

MPU6050	VCC / GND / SCL / SDA / AD0 / INT	1	Powered at 3.3V; SCL→GPIO22, SDA→GPIO21; AD0→GND (addr 0x68); INT unused
DS3231 RTC	VCC / GND / SCL / SDA / BAT / SQW	1	Shares I2C bus with MPU6050; CR2032 battery backup; SQW unused
NEO-6M GPS	VCC / GND / TX / RX / PPS	1	UART2 at 9600 baud; TX→GPIO16 (RX2), RX→GPIO17 (TX2); PPS unused
Pull-up resistors	R1 (SDA), R2 (SCL)	2	4.7 kΩ each, tied between 3.3V and the SDA/SCL lines

3.6. Communication Protocols

- WiFi (TCP/IP): Main transport on the 2.4 GHz band connecting all nodes.
- HTTP/MJPEG: Infinite multipart video stream from the Vision Node to the Python hub.
- HTTP/REST (GET): Event-triggered JSON query from the Python hub to the Sensor Node.
- I2C: Inter-Integrated Circuit protocol for the MPU6050 and DS3231 on the Sensor Node.
- UART: Asynchronous serial protocol for NMEA sentence reception from the NEO-6M.

3.7. Data Flow

The end-to-end pipeline for a single pothole event proceeds as follows:

1. The ESP32-CAM continuously pushes 320×240 JPEG frames over WiFi.
2. The Python hub decodes each frame into a NumPy array and runs YOLOv8 inference.
3. The SORT tracker assigns a persistent ID to each detected pothole.
4. When a tracked pothole's centroid crosses the reference line (set at 75% of frame height), the system fires an HTTP GET to the Sensor Node.
5. The Sensor Node reads the MPU6050 via I2C, retrieves the latest GPS NMEA sentence, and returns a JSON payload.
6. Python calculates the severity index, writes the record to a CSV file, and updates the video overlay.

4. FIRMWARE IMPLEMENTATION

4.1. Vision Node Firmware (*esp_32_cam_final.ino*)

The camera firmware uses the `esp_http_server` library to implement a persistent HTTP server. The `stream_handler` function runs a tight loop: `esp_camera_fb_get()` acquires the latest frame buffer from PSRAM, the raw JPEG bytes go out as an HTTP chunk with a Content-Length header, and `esp_camera_fb_return()` immediately recycles the buffer back to the hardware pool. Failing to call the return function exhausts PSRAM within milliseconds and causes a kernel panic.

Reliability mechanisms include a hardware Watchdog Timer (WDT) that resets the microcontroller if the WiFi stack hangs under thermal load. Additionally, a blocking WiFi state machine prevents the HTTP server from starting until a valid DHCP lease is obtained.

4.2. Sensor Node Firmware (*esp_32_final.ino*)

The sensor node firmware follows an event-triggered sensing model. The `handleQuery()` HTTP handler is the sole point where hardware reads occur. Specifically, when called, it:

1. Performs a 6-byte I2C burst read from the MPU6050 starting at register 0x3B, acquiring all three 16-bit acceleration register pairs simultaneously to prevent timing mismatches between axes.
2. Scales raw LSB values to SI units: $a \text{ (m/s}^2\text{)} = (\text{raw} / 16384) \times 9.81$.
3. Retrieves the latest valid NMEA \$GPRMC sentence from the UART buffer and parses latitude and longitude.
4. Reads the DS3231 RTC via I2C for the ISO 8601 timestamp.
5. Assembles and returns the JSON payload.

4.3. WiFi Credential Management

WiFi credentials are managed through a centralized configuration mechanism that injects the network SSID and password into both firmware source files before flashing, ensuring both nodes always share the same network credentials without manual source code editing.

4.4. MPU6050 Calibration

A dedicated calibration script zeroes out the sensor’s resting state before deployment. The script discards the first 100 readings to allow physical settling, then averages 1000 reads across all six axes to establish the vehicle’s baseline vector. A proportional control loop then iteratively adjusts the MPU6050 hardware offset registers via `accelgyro.setXAccelOffset()` until all axes read within eight LSB of their target values (zero for X/Y, 16384 for Z gravity). The resulting calibration offsets, which the engineer hard-codes into the Sensor Node firmware, ensure that `J_peak` reflects only genuine vertical impulse events rather than gravitational bias or vehicle tilt.

5. COMPUTER VISION AND TRACKING PIPELINE

5.1. YOLOv8 Model Training and Performance

The IPDS uses a custom-trained YOLOv8m model accessed through the Ultralytics Python library [15]. The model was trained on a publicly available pothole-detection dataset sourced from Kaggle, originally annotated in Pascal VOC XML format and converted to YOLO format for training. Table 4 summarizes the measured detection metrics, and Table 5 reports the dataset composition.

Table 4. YOLOv8m model performance on the validation split.

Metric	Value
mAP@0.5	81.68%
mAP@0.5:0.95	55.95%
Precision	82.33%
Recall	74.42%

Table 5. Dataset composition and split used for YOLOv8m training.

Parameter	Value
Dataset Source	Kaggle Pothole Detection Dataset (Andrew MVD)
Total Images	665
Training Images	598 (90%)
Validation Images	67 (10%)
Number of Classes	1 (pothole)
Original Annotation Format	Pascal VOC (XML)
Converted Annotation Format	YOLO (normalized TXT)
Training Input Resolution	640 x 640

No separate held-out test split was carved out for this dataset; the reported metrics in Table 4 were computed on the validation split shown above, consistent with the standard Ultralytics train/val workflow. The dataset license follows the original Kaggle dataset's terms.

The system selects YOLOv8 over older architectures (YOLOv3, YOLOv5) for several reasons. The anchor-free detection head removes the need for manual anchor box tuning, the decoupled head structure separates classification from regression, and native ONNX and TorchScript export enables future deployment to edge inference engines. On a CPU-only host, the trained model achieves 10–15 FPS on 320×240 input resolution, sufficient for vehicle speeds up to 60 km/h. The 82.33% precision confirms that most bounding boxes the model draws correspond to genuine potholes, while the 74.42% recall indicates occasional missed detections under challenging conditions such as shadows, wet pavement, and glare. The multi-stage false positive suppression pipeline and sensor fusion layer compensate for these missed detections.

5.2. Detection Pipeline

The detection pipeline operates as follows:

1. cv2.VideoCapture(mjpeg_url) continuously decodes the MJPEG stream into BGR NumPy arrays.
2. The Ultralytics model wrapper automatically letterbox-pads and normalizes the frame to the model's input tensor dimensions.
3. Detections below CONF_THRESHOLD = 0.25 are immediately discarded.
4. Remaining bounding boxes pass to the SORT tracker.

5.3. SORT Multi-Object Tracking

The Simple Online and Realtime Tracking algorithm [14] maintains consistent object identity across video frames. For each frame:

1. Each active Kalman filter predicts the expected bounding box position based on velocity state.
2. The Hungarian algorithm minimizes the global Intersection-over-Union (IoU) cost between predicted tracks and new detections.

3. Tracks with no matching detection for MAX_AGE consecutive frames terminate; new tracks initialize for unmatched detections that survive MIN_HITS consecutive frames.

Consequently, the same physical pothole retains the same ID from initial detection until it crosses the trigger line, thereby preventing duplicate sensor queries and duplicate CSV entries.

5.4. Geometric Heuristic False-Positive Suppression

Three geometric filters apply to each tracked bounding box before the system authorizes a sensor query.

Area Filter: Bounding boxes exceeding 25% of total frame area indicate large-scale road features such as tunnel entrances or vehicle underbodies; the system discards them:

$$area_ratio = [(x_2 - x_1)(y_2 - y_1)] / (W_frame \times H_frame) \leq 0.25 \quad (2)$$

Aspect Ratio Filter: Bounding boxes with a width-to-height ratio above 3.0 indicate road markings, tar cracks, or speed bumps; the pipeline rejects them:

$$aspect_ratio = (x_2 - x_1) / (y_2 - y_1) \leq 3.0 \quad (3)$$

Persistence Filter: If a tracked centroid remains stationary in the frame for more than 10 consecutive frames while the vehicle is moving, it indicates a static scene element such as a parked vehicle shadow or road marking; the system therefore rejects it.

5.5. Reference Line Trigger

The trigger mechanism monitors the vertical position of each tracked centroid relative to a reference line at 75% of frame height, which corresponds to approximately 1.0 metre in front of the vehicle's front axle at the recommended -35° camera pitch angle. When a pothole centroid crosses this line, the event-driven HTTP query fires. At typical deployment speeds, approximately 37 ms of travel time remain before tyre impact, providing sufficient margin for the WiFi HTTP round-trip latency.

6. SENSOR FUSION AND SEVERITY MODEL

6.1. Fusion Methodology

The IPDS employs a vision-dominant sensor fusion model. Computer vision provides the primary hypothesis ("a pothole is present"); the accelerometer, conversely, provides physical validation ("the vehicle experienced an impact"). Together, these signals form a logical AND gate: both must confirm the event before the system creates a log entry. This design eliminates two categories of false positives:

- Visual false positives (shadows, manhole covers, tar snakes): YOLO detects them, but they produce no accelerometer spike. The fusion layer discards these events.
- Inertial false positives (speed bumps, railroad tracks, emergency braking): these produce an accelerometer spike but trigger no YOLO detection. The system ignores them as well.

6.2. Peak Jerk Calculation

Raw accelerometer data converts from 16-bit LSB values to physical acceleration as follows:

$$a_r = (raw_r / 16384) \times 9.81 \text{ [m/s}^2\text{]} \quad (4)$$

Peak jerk (rate of change of acceleration) computes as:

$$J_{peak} = \max(|\Delta a_r / \Delta t|) \text{ [m/s}^3\text{]} \quad (5)$$

Following calibration, J_{peak} reflects only genuine vertical impulse events because the hardware offset registers zero out gravitational and tilt contributions.

6.3. Severity Index

To produce a single, comparable hazard rating, the normalized severity index S combines the YOLO confidence score $C_{yolo} \in [0, 1]$ and a normalized jerk magnitude J_{norm} as a weighted linear combination:

$$S = w_1 \cdot C_{yolo} + w_2 \cdot J_{norm} \quad (6)$$

where $w_1 = 0.7$ and $w_2 = 0.3$ are heuristic weights selected during preliminary testing. The higher weight on the visual channel reflects its superior precision (82.33% vs. 74.42% recall). J_{norm} computes as:

$$J_{norm} = \min(J_{peak} / J_{max}, 1.0) \quad (7)$$

where J_{max} is a calibrated saturation threshold corresponding to the maximum jerk from the worst-case road surface. The resulting $S \in [0, 1]$ provides a dimensionless, cross-vehicle-comparable hazard severity rating.

6.4. CSV Log Schema

Each confirmed pothole event appends to a structured CSV log file with eleven fields: date, time, frame_id, pothole_id, confidence, bounding_box_area, aspect_ratio, peak_jerk, severity, latitude, longitude.

The date and time fields store the DS3231 RTC timestamp split into ISO 8601 components for easier database partitioning. frame_id is the sequential frame counter from the MJPEG stream, enabling post-hoc correlation of log entries with the annotated video file. pothole_id is the persistent SORT tracker ID, guaranteeing that a single physical pothole generates exactly one log row regardless of how many frames it was tracked across. confidence is the raw YOLOv8 output score C_{yolo} . bounding_box_area and aspect_ratio retain the geometric filter values computed at trigger time for offline audit of filter decisions. peak_jerk is J_{peak} in m/s^3 , severity is the fused index $S \in [0, 1]$, and latitude/longitude are the WGS-84 decimal-degree coordinates parsed from the NEO-6M NMEA sentence. This schema imports directly into Firebase Firestore, PostGIS, or any GIS platform for geospatial heatmap visualization and municipal maintenance prioritization.

7. RESULTS AND EVALUATION

7.1. Detection Performance

The YOLOv8m model underwent training on the Kaggle pothole-detection dataset described in Table 5. As Table 4 in Section 5 reports, the model achieves 81.68% mAP@0.5 on the validation split. This figure is competitive with contemporary CPU-deployable pothole detectors reported in the literature [4,5]. The mAP@0.5:0.95 of 55.95% indicates that bounding box localization precision degrades under stricter IoU thresholds, which is consistent with the inherent geometric variability of pothole shapes described in Section 2.

In live deployment tests, the system maintained 10–15 FPS inference throughput on a CPU-only laptop host. End-to-end latency from frame capture to CSV write measured approximately 170–200 ms. Consequently, at a vehicle speed of 40 km/h, the spatial detection error stays under 2 metres, well within the NEO-6M GPS uncertainty envelope of 2.5–5.0 metres.

7.2. False Positive Suppression Efficacy

The multi-stage false positive suppression pipeline underwent evaluation against a set of known false positive scenarios. The pipeline is structured as a logical AND gate across four sequential, independent stages: (1) a confidence threshold filter that discards YOLOv8 detections below 0.25; (2) a SORT tracking filter that discards objects failing to persist across MIN_HITS consecutive frames; (3) three geometric heuristic sub-filters for area, aspect ratio, and persistence; and (4) the sensor fusion gate, which cross-validates the visual detection against a physical accelerometer jerk spike from the MPU6050. A single rejection at any stage terminates the detection event immediately without triggering the sensor node, conserving bandwidth and preventing spurious CSV log entries.

This layered approach exploits the statistical independence of failure modes across modalities: a shadow may fool the camera but will not generate a jerk spike, while a speed bump will generate a jerk spike but will not produce a YOLO detection. The probability of both modalities being deceived by the same non-hazard simultaneously is extremely low, making the AND-gate fusion highly effective. Table 6 summarizes the outcome for each tested scenario, demonstrating that each filter stage correctly handles its target class of false positive.

Table 6. False positive suppression outcomes by filter stage.

Scenario	Primary Filter	Outcome
Road shadow	Persistence Filter	Rejected (centroid stationary) 15/15 trials
Manhole cover	Sensor Fusion Gate	Rejected (no jerk spike) 15/15 trials
Speed bump	Sensor Fusion Gate	Rejected (no YOLO detection) 20/20 trials
Tar crack / stripe	Aspect Ratio Filter	Rejected (AR > 3.0) 20/20 trials
Large vehicle / tunnel	Area Filter	Rejected (area > 25% of frame) 10/10 trials
Railroad crossing	Sensor Fusion Gate	Rejected (no YOLO detection) 10/10 trials
Confirmed pothole	All stages passed	Logged 10/10 trials

In controlled field evaluation, the multi-stage pipeline successfully suppressed all tested false positive categories across 90 total trials (15 road shadows, 15 manhole covers, 20 speed bumps, 20 tar cracks, 10 large vehicle/tunnel entries, and 10 railroad crossings). Every scenario was correctly rejected at its designated filter stage. All 10 confirmed pothole events were correctly passed through all stages and logged with their associated GPS coordinates, RTC timestamp, and calculated severity index.

7.3. System Limitations

Field evaluation revealed several limitations that future work must address.

GPS Precision: The NEO-6M civilian GPS module carries a positional error of 2.5–5.0 metres. While acceptable for road-level mapping, lane-level localization requires RTK-corrected GPS.

Night Operation: The OV2640 performs poorly in low-light conditions. High ISO noise reduces YOLO confidence scores below the 0.25 threshold, causing missed detections. Consequently, the 74.42% recall figure is likely to decline further in night conditions; supplemental IR illumination is advisable for around-the-clock deployment.

Suspension Variability: The relationship between pothole depth and peak jerk magnitude is highly vehicle-specific. The severity formula assumes a fixed vehicle suspension transfer function; fleet-wide use therefore requires per-vehicle calibration runs using the included MPU6050 calibration script.

WiFi Dependency: The current architecture requires all three nodes to share a common 2.4 GHz WiFi network. In remote deployment environments, this constraint is limiting; LoRaWAN integration is a priority for future work.

Processing Bottleneck: Running YOLOv8 on a CPU-only host caps throughput at 15 FPS. GPU-accelerated hosts such as the NVIDIA Jetson Nano would lift this ceiling and support higher vehicle speeds and larger input resolutions.

Known limitations of the suppression model specifically: flooded potholes present an additional edge case, since standing water significantly alters the visual texture of a pothole, sometimes reducing confidence scores and causing premature Stage-1 rejection.

7.4. Scalability

Because the Sensor Node exposes a standard REST API, the architecture scales horizontally. For instance, multiple camera nodes (front, left, right) can independently stream to a single processing hub and each query the shared Sensor Node as needed. Upgrading the Vision Node to a stereoscopic depth camera requires only pointing the Python script to the new MJPEG stream URL, with no firmware changes necessary.

8. FUTURE WORK

Edge AI Migration: Deploying a quantized TensorFlow Lite Micro version of the YOLOv8 model directly onto an ESP32-S3 or ESP32-P4, removing the laptop processing hub entirely.

Cloud Dashboard: Building a Next.js and Firebase frontend to display CSV log data as real-time geospatial heatmaps for municipal road maintenance authorities.

LoRaWAN Integration: Replacing the WiFi communication layer with a LoRa radio module (e.g., SX1276) for deployment in rural environments without WiFi infrastructure.

Fleet Normalization: Implementing a per-vehicle suspension calibration model to normalize jerk magnitude across mixed vehicle fleets, improving cross-platform consistency of the severity index.

Depth Estimation: Integrating a stereoscopic camera or LiDAR module to estimate absolute pothole depth, adding a measurable physical dimension to the severity calculation beyond the inertial proxy.

Dataset Expansion: Expanding the training dataset with additional annotated images from diverse geographic regions, weather conditions (rain, fog, night), and road surface types to push recall beyond the current 74.42% and close the gap toward the 82.33% precision figure.

Improved Hardware: Replacing the OV2640 with a wide-dynamic-range image sensor and the MPU6050 with an automotive-grade IMU (e.g., Bosch BMI series) to improve night-time detection and reduce inertial noise floors.

9. CONCLUSIONS

This paper presented the Intelligent Pothole Detection System (IPDS), a vehicle-mounted IoT platform for automated, real-time road hazard assessment. The two central technical contributions are a dual ESP32 hardware architecture that resolves the concurrency conflict between MJPEG video streaming and I2C inertial sensing, and a vision-dominant sensor fusion model that chains YOLOv8 computer vision with MPU6050 accelerometry through an event-driven HTTP protocol to achieve robust, multi-modal false positive rejection.

The custom-trained YOLOv8m model achieves 81.68% mAP@0.5, 82.33% precision, and 74.42% recall on the validation split, confirming its suitability for CPU-bound mobile deployment. The system produces geo-tagged, timestamped, severity-indexed CSV logs at a hardware cost under Rs. 2,500 per unit, one to two orders of magnitude below conventional road survey equipment. Field evaluation confirmed real-time operation at 10–15 FPS and end-to-end event latency of approximately 170–200 ms. The severity index combines the YOLO confidence score ($w_1 = 0.7$) and normalized peak jerk magnitude ($w_2 = 0.3$), providing a dimensionless, cross-platform-comparable hazard rating suitable for municipal maintenance prioritization.

The results suggest that low-cost vision-assisted IoT platforms may provide a practical approach for scalable road-condition monitoring. As more vehicles are equipped with systems of this type, the aggregate data stream has the potential to shift municipal road maintenance from costly reactive patching to efficient, data-driven proactive resurfacing, thereby reducing economic waste, improving road safety, and contributing to smarter city infrastructure.

10. AUTHOR CONTRIBUTIONS

Conceptualization, S.T.; methodology, S.T.; software, S.T.; validation, S.T., S.B., E.K. and S.S.; formal analysis, S.T.; investigation, S.T., S.B., E.K. and S.S.; writing: original draft preparation, S.T.; writing: review and editing, S.T., S.B., E.K. and S.S. All authors have read and agreed to the published version of the manuscript.

11. FUNDING

This research received no external funding.

12. DATA AVAILABILITY STATEMENT

The firmware source code (ESP32-CAM and Sensor Node), Python detection pipeline, KiCad hardware schematics, trained YOLOv8m model weights (pothole_yolov8.pt), and a sample field-test output log are

openly available at <https://github.com/SanTiwari07/PotHoleDetection> under an open-source license. The training dataset is publicly available at <https://www.kaggle.com/datasets/andrewmvd/pothole-detection> (Andrew MVD, Kaggle).

13. ACKNOWLEDGMENTS

The authors thank the open-source communities behind the Ultralytics YOLOv8, FilterPy, and OpenCV projects, without which this work would not have been possible.

14. AI USE STATEMENT

Large language model tools were used for language refinement and manuscript editing. All technical content, experimental results, analysis, and conclusions were verified by the authors, who take full responsibility for the final manuscript.

15. CONFLICTS OF INTEREST

The authors declare no conflicts of interest.

REFERENCES

- [1] Eriksson, J.; Girod, L.; Hull, B.; Newton, R.; Madden, S.; Balakrishnan, H. The Pothole Patrol: Using a Mobile Sensor Network for Road Surface Monitoring. In Proceedings of the 6th International Conference on Mobile Systems, Applications, and Services (MobiSys), Breckenridge, CO, USA, 17–20 June 2008; pp. 29–39.
- [2] Mohan, A.; Poobal, S. Crack Detection Using Image Processing: A Critical Review and Analysis. *Alexandria Engineering Journal* 2018, 57, 787–798.
- [3] Fan, R.; Ozgunalp, U.; Hosking, B.; Liu, M.; Pitas, I. Pothole Detection Based on Disparity Transformation and Road Surface Modeling. *IEEE Transactions on Image Processing* 2020, 29, 897–908.
- [4] Saluky; et al. Pothole Detection on Urban Roads Using YOLOv8. In Proceedings of the IEEE International Conference on Information Technology and Computing, 2023.
- [5] Alzamzami, O.; et al. Enhanced Pothole Detection in Road Condition Assessment Using YOLOv8. In Proceedings of the IEEE International Conference for Women in Innovation, Technology and Entrepreneurship (ICWITE), 2024; pp. 1–6.
- [6] Yurdakul, M.; Guller, M.K. An Enhanced YOLOv8 Model for Real-Time and Accurate Pothole Detection and Measurement. *arXiv preprint arXiv:2505.04207*, 2025.
- [7] Mednis, A.; Strazdins, G.; Zviedris, R.; Kanonirs, G.; Selavo, L. Real Time Pothole Detection Using Android Smartphones with Accelerometers. In Proceedings of the 2011 International Conference on Distributed Computing in Sensor Systems (DCOSS), Barcelona, Spain, 27–29 June 2011; pp. 1–6.
- [8] Bhoraskar, S.; Vankadhara, N.; Raman, B.; Kulkarni, P. Wolverine: Traffic and Road Condition Estimation Using Smartphone Sensors. In Proceedings of the 2012 4th International Conference on Communication Systems and Networks (COMSNETS), Bangalore, India, 3–7 January 2012.

- [9] Shtayat, T.; et al. Accelerometer Sensor Network for Reliable Pothole Detection. In Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics (SMC), 2021.
- [10] Koch, C.; Brilakis, I. Pothole Detection in Asphalt Pavement Images. *Advanced Engineering Informatics* 2011, 25, 507–515.
- [11] Tedeschi, A.; Benedetto, F. A Real-Time Automatic Pavement Crack and Pothole Recognition System for Mobile Android-Based Devices. *Advanced Engineering Informatics* 2017, 32, 11–25.
- [12] Samsir, S.; et al. Method for Detecting Rough Road Index Using IoT Sensor Fusion and V2V Interaction for Efficient Road Infrastructure Management. *PLOS ONE* 2024.
- [13] Wang, C.; Xu, Z.; Gao, J. Road Damage Detection Using Deep Learning Approaches: A Review. *IEEE Access* 2022, 10, 1–16.
- [14] Bewley, A.; Ge, Z.; Ott, L.; Ramos, F.; Upcroft, B. Simple Online and Realtime Tracking. In Proceedings of the 2016 IEEE International Conference on Image Processing (ICIP), Phoenix, AZ, USA, 25–28 September 2016; pp. 3464–3468.
- [15] Jocher, G.; et al. Ultralytics YOLOv8 Documentation, 2023. Available online: <https://docs.ultralytics.com> (accessed on 14 June 2026).

Disclaimer/Publisher's Note: The statements, opinions, and data contained in this preprint are solely those of the individual authors and do not represent the views of Preprints.org or any affiliated journal or publisher.